Learning algorithms

Meeting 1. Models, data, and validation

Course meeting 21 · November 4, 2026

We learn from experience, and we use what we have learned to make predictions. Dark clouds suggest rain. A familiar sentence suggests its missing word. Sometimes we can state the rule behind a prediction; sometimes we recognize a pattern without being able to explain exactly how. Learning algorithms turn part of this familiar process into something we can specify, run, and evaluate.

The central question of this unit is how to build machines that learn from examples. We begin with the ideas needed to tell whether learning has occurred: a model, a prediction task, a fair evaluation, and a meaningful comparison.

The main points

1. **Learning produces a model; prediction uses it.**
   The learning algorithm works with examples and their answers. The fitted model then receives new inputs and produces predictions.
2. **A feature must be available when the prediction is needed.**
   A completed historical record may contain information that did not exist at prediction time. Define the task before selecting columns.
3. **Validation rehearses prediction on unseen cases.**
   Fit on training data, choose among candidates with validation data, and assess the final choice on a separate test set.
4. **Start with a simple baseline.**
   A training mean or yesterday's count gives a concrete reference. A sophisticated model must earn its usefulness through comparison.
5. **The error measure affects the comparison.**
   MAE and MSE prevent positive and negative misses from cancelling, but assign different costs to large mistakes.
6. **A closer training fit need not predict better.**
   Use validation to choose flexibility. Keep unavailable information and held-out answers out of model development.

1. Models, learning, and prediction

Prediction is evidence of learning

A **model** is a rule or representation that maps inputs to predictions. It can be as simple as a constant, or as complicated as a large neural network. A **learning algorithm** is the procedure that constructs or adjusts that model using data. The procedure and its result are different things.

Human learning provides a useful analogy. We accumulate examples, remember some of them, and find patterns that let us respond to new situations. Learning can involve memorization, but remembering a particular answer does not by itself establish that we can handle a new case. To look for that broader ability, we examine predictions on examples the learner has not already studied.

> **The missing word.**
>
> The class compared completing a Greek sentence with completing an English one. The Greek prompt was Ο ήλιος ανατέλλει από την _____. Its completion, ανατολή, means "east": the sun rises in the east. The English prompt was "The sun sets in the _____," completed by "west." Familiarity with the language makes the clues usable. Recognizing that something is a language is less knowledge than being able to predict its missing word.

Here, prediction means inferring a hidden answer; it need not concern a future event. Forecasting tomorrow's demand is one prediction task. Recognizing an object in an existing photograph is another. In both cases, a useful evaluation asks the model to supply an answer that was not given to it.

Two phases: fit, then predict

Learning
Examples + answers → learning algorithm → fitted model

Prediction
New input → fitted model → prediction

During **fitting**, the algorithm uses the training examples and their known answers. During **prediction**, we supply inputs to the fitted model. The answers are what we want it to produce. For the examples here, prediction leaves the fitted model unchanged.

The scikit-learn library expresses these two phases through a common interface:

```
model.fit(X_train, y_train)
predictions = model.predict(X_new)
```

The first call learns from inputs and targets together. The second call receives only the new inputs. Many different model families use these same method names, even though the algorithms inside `fit` differ. Human learning and prediction often overlap; separating them explicitly makes a machine-learning experiment easier to reason about.

2. Data and the prediction boundary

Observations, features, and targets

A dataset records observations. In a table, one row usually represents one example, and columns record its attributes. The **features** are the attributes available to the predictor. We collect them in a matrix X. The **target** is the answer we want to predict; the corresponding recorded answers form y. A hat distinguishes a prediction $\hat{y}$ from an observed answer y.

Our running example is daily bike-rental demand. At the start of a day, we want to predict that day's total rentals. We assume the preceding day's total has already been finalized. Two available features are yesterday's rental count and whether today is a working day, a calendar fact known in advance.

Three daily records from the bike example

Date	Yesterday's rentals feature	Working day feature	Today's total target
Jan. 1, 2012	2,485	0	2,294
Jan. 2, 2012	2,294	0	1,951
Jan. 3, 2012	1,951	1	2,236

For these three rows, X has shape 3×2: three examples and two features. The date orders the records. The target column is available in the historical file because the days have ended; it would not be available when making the morning prediction.

A column's presence in a file does not make it a valid feature. Today's final casual-rider and registered-rider counts are unavailable at the start of the day. So is today's observed daily weather. A weather forecast issued beforehand could be available, but a forecast and the subsequently observed weather are different pieces of information.

Regression and classification

Regression predicts a numerical amount, such as a rental count or a temperature. A count is integer-valued, but predicting its amount is still a regression task. **Classification** predicts a category: up or down, cold or warm, cat or dog. We could turn the rental question into classification by asking whether demand will exceed 5,000 rentals. That changes the target and the way we assess errors.

Terminology can be overloaded. In particular, logistic regression is commonly used for classification despite its name. Identify the task by the answer being predicted, rather than by a word appearing in an algorithm's name.

What changes with time?

The class considered weather, stock prices, movie ratings, and physical measurements. These do not all change in the same way. Preferences and demand can shift with trends; a physical relationship may remain stable over the period of interest. Even without a date column, observations have a collection context, and a model's usefulness depends on whether the relationships it learned still apply.

Imagine a red line marking the present. Behind it are observations we can use; ahead of it are outcomes not yet known. For a forecasting task, our evaluation should respect that boundary. A successful fit to old demand is not a guarantee of successful prediction after demand changes.

3. Validation as a rehearsal

How can we assess future predictions before the future arrives? We set aside some observed cases before fitting, treat their answers as hidden, and ask the model to predict them. Then we reveal the recorded answers and measure the errors. **Validation** here means evaluating a model during development, rather than checking whether a data file contains valid entries.

Hiding an answer means keeping it out of the fitting procedure, not merely removing it from the final call to `predict`. A model that already learned from that answer has not encountered an unseen case.

Set	Purpose	How it is used
Training	Learn the model.	Fit its weights, rules, or stored quantities.
Validation	Compare candidates.	Choose features, model family, or settings.
Test	Assess the final choice.	Evaluate after development choices are fixed.

Studying, practice, and a first job.

Classes and assignments resemble training. Practice cases with answers withheld resemble validation. A first job suggests the challenge of deployment. The machine-learning test set is a final reserved rehearsal before deployment: its answers already exist, but must not guide model selection. Both validation and test sets have recorded answers; their different roles are what distinguish them.

Split in the order the task requires

Suppose a forecast is needed on January 2. If our experiment fits the model using January 3 outcomes and then evaluates it on January 2, the fitted model has information that could not have been available at the intended prediction time. A low error in that experiment does not establish a good one-day-ahead forecast.



Earlier → later. The first source day is omitted because yesterday's count is unavailable, leaving 730 usable daily examples.

The bike experiment therefore trains on earlier dates and validates on later ones, while reserving the final period for testing. Randomly mixing dates would answer a different question. A random split can be suitable for independent examples from the same population; it is not a universal rule for forecasting.

Here we make successive one-day-ahead predictions with a fixed model. Once a validation day ends, its observed total becomes available as the next day's "yesterday" feature. That is allowed by our task. The fitted weights or stored training mean do not change. This experiment is different from predicting six months ahead without seeing any intervening outcomes.

Validation can be consulted while developing several candidates. The final test set remains separate because those comparisons influence our decisions. If we keep trying models until one has a good test score, we have used that set for model selection. It no longer provides the same independent final assessment.

4. Start with a baseline

An error value needs a reference. Before celebrating a complicated model, ask what a very simple rule can do on the same task. A **baseline** is that reference model. It need not be a horizontal line, and it is not a guarantee of acceptable performance.

Always predict the training mean

For regression, one simple rule predicts the average of the training targets for every input:

$$\bar{y}_{\text{train}} = \frac{1}{n_{\text{train}}} \sum_{i=1}^{n_{\text{train}}} y_i, \qquad \hat{y} = \bar{y}_{\text{train}}.$$

In words: The training mean, y-bar subscript train, equals the sum of the training target values divided by the number of training examples. The prediction, y-hat, equals that training mean for every input.

The bike training mean is **3,412.4 rentals**. This model has learned one number. Its prediction is the same for a working day or a holiday, and for low or high demand yesterday. On a plot, it is a horizontal line. Other models must be compared against that concrete, reproducible starting point.

Why the mean?

Use four small training targets: 2, 4, 6, and 8. If we always predict a constant c, the mean squared error is

$$\text{MSE}(c) = \frac{(2-c)^2 + (4-c)^2 + (6-c)^2 + (8-c)^2}{4} = 5 + (c-5)^2.$$

In words: The mean squared error for constant c equals the sum of the squares of 2 minus c, 4 minus c, 6 minus c, and 8 minus c, divided by 4. This simplifies to 5 plus the square of c minus 5.

The minimum occurs at $c = 5$, the training mean. Moving the line either above or below 5 increases the squared error. This establishes the best constant for these training observations under squared error; it does not establish the best model for every future observation.

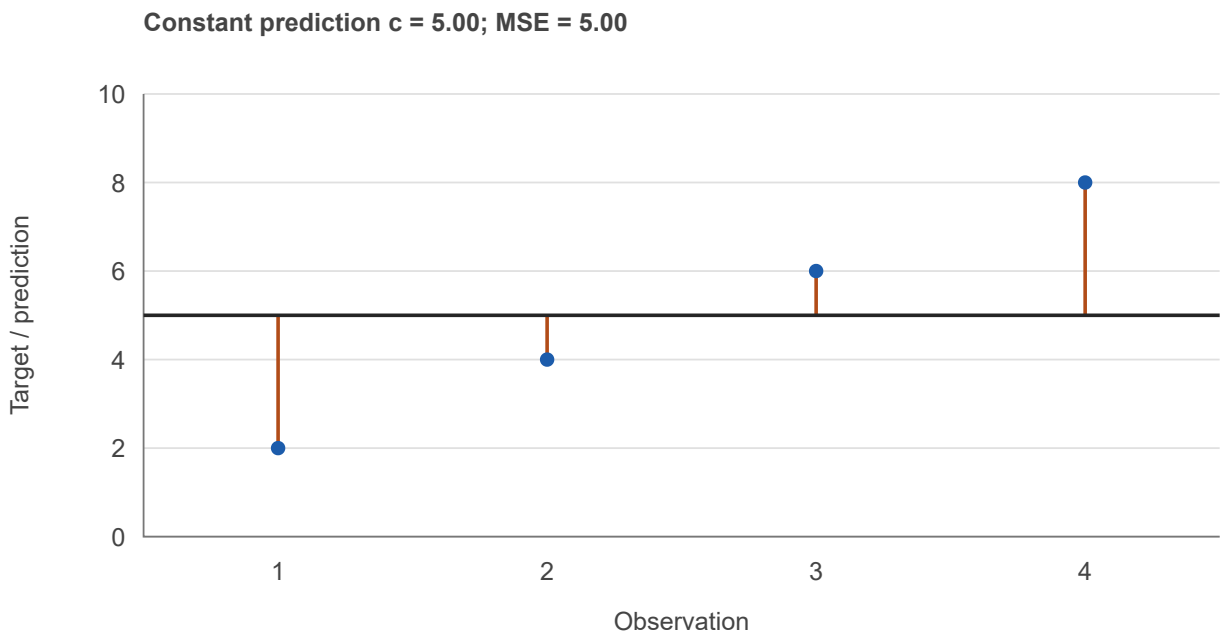


Figure description: The horizontal axis lists four observations; the vertical axis runs from zero to ten. The target points are 2, 4, 6, and 8. A horizontal line predicts 5.00 for every observation. The signed residuals, target minus prediction, are -3.00, -1.00, 1.00, 3.00. Vertical segments join each target to the prediction line. Mean squared error is 5.00. The smallest possible value is 5, attained when the line is at c equals 5.

Four invented targets, separate from the bike data. Vertical segments show the residuals. The mean minimizes their average squared length.

Why this holds for any training sample

Writing each residual as $y_i - c = (y_i - \bar{y}) + (\bar{y} - c)$ and expanding gives

$$\frac{1}{n}\sum_i (y_i - c)^2 = \frac{1}{n}\sum_i (y_i - \bar{y})^2 + (c - \bar{y})^2 .$$

In words: The average squared distance of the targets from c equals their average squared distance from the sample mean, plus the squared distance between c and the sample mean.

The cross term vanishes because $\sum_i (y_i - \bar{y}) = 0$. Only the last term depends on c , and it is minimized at the mean. For absolute error, a median is an optimal constant instead.

Classification baselines depend on the classes

The class also considered predicting whether tomorrow's temperature will go up or down. With two equally likely outcomes, uninformed guessing has expected accuracy 50%. That reference changes when the outcomes are imbalanced. If a hypothetical dataset contains 99% negative cases, always predicting "negative" achieves 99% accuracy while missing every positive case.

A high accuracy can therefore coexist with failure on the cases that matter. The medical example in class illustrated this imbalance; 99% was a hypothetical proportion, not a measured medical rate. Compare with the relevant class frequencies and consider which mistakes matter for the task.

5. Measuring the misses

For observation i, define the **residual** by $e_i = y_i - \hat{y}_i$. A positive residual means the model predicted too little; a negative one means it predicted too much. Simply averaging signed residuals can hide large errors. If the observed answers are 2 and 6 and the model predicts 4 for each, the residuals are -2 and $+2$: their mean is zero, although both predictions are wrong.

Two common measures avoid cancellation:

$$\text{MAE} = \frac{1}{n}\sum_{i=1}^{n} |\, y_i - \hat{y}_i \,|, \qquad \text{MSE} = \frac{1}{n}\sum_{i=1}^{n} (y_i - \hat{y}_i)^2.$$

In words: Mean absolute error equals the sum, over n examples, of the absolute difference between each observed target and its prediction, divided by n. Mean squared error equals the sum of the squared differences, divided by n.

Mean absolute error (MAE) averages the sizes of the misses. **Mean squared error (MSE)** averages their squares. Both are nonnegative. For the two residuals above, MAE is 2 and MSE is 4.

Doubling a miss doubles its contribution to MAE but quadruples its contribution to MSE. Squared error therefore gives relatively more weight to large misses. MAE has the units of the target, rentals here; MSE has squared target units. Their numerical magnitudes cannot be compared as if they were the same quantity.

Why not use a fourth power?

This question arose in class after the suggestion that squared error is useful because it penalizes large mistakes more. Larger powers would emphasize extreme misses even more strongly. The penalty should express what matters for the application, and its mathematical form also affects how easily we can fit a model.

Squared error is smooth. For a constant predictor it gives the simple formula above; for a linear model it gives a well-studied least-squares problem. Absolute error has a corner at zero, so ordinary differentiation needs modification there. It can still be minimized using appropriate methods. Neither squared error nor absolute error is universally best, and using squared error does not guarantee that every model's training problem is easy.

The loss used to fit a model and the metric used to compare its predictions can differ. A model fitted by minimizing squared error can still be evaluated using MAE. State the comparison criterion before choosing a winner, rather than selecting whichever metric favors a preferred model.

6. Fit, predict, and compare

We can now put the pieces together. Initialize a candidate, fit it using the training inputs and targets, predict from validation inputs, and compare those predictions with the validation answers. The interface stays the same when the candidate model changes.

The mean baseline in scikit-learn

Assume the daily table has been sorted, yesterday's count constructed, and the training and validation periods separated. The two feature columns are `yesterday_rentals` and `workingday`; the target is `cnt`.

```
from sklearn.dummy import DummyRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error

features = ["yesterday_rentals", "workingday"]
X_train, y_train = train[features], train["cnt"]
X_valid, y_valid = valid[features], valid["cnt"]

model = DummyRegressor(strategy="mean")
model.fit(X_train, y_train)
predictions = model.predict(X_valid)

print(mean_absolute_error(y_valid, predictions))
print(mean_squared_error(y_valid, predictions))
```

OUTPUT

```
2048.415771
5978763.960663
```

The baseline learns from `y_train`. It accepts the feature matrix to follow the common interface, but its prediction does not depend on feature values. Validation answers enter only in the evaluation calls.

Could yesterday be a better guide?

If asked for tomorrow's temperature, we might predict that it will be the same as today's. For bike demand the corresponding rule is $\hat{y}_t = y_{t-1}$: predict yesterday's count. This **persistence baseline** responds to recent demand without fitting any coefficients. Plotting it against the observations amounts to shifting the observed series by one day.

The third candidate is a fitted linear model using yesterday's rentals and the working-day flag. Its prediction has the form

$$\hat{y} = b + w_1 \,\text{yesterday's rentals} + w_2 \,\text{working-day flag}.$$

In words: The prediction equals intercept b, plus weight w one times yesterday's rental count, plus weight w two times the working-day flag.

The coefficients and intercept are fitted on training data. We do not need the fitting derivation to compare its validation predictions with the two baselines.

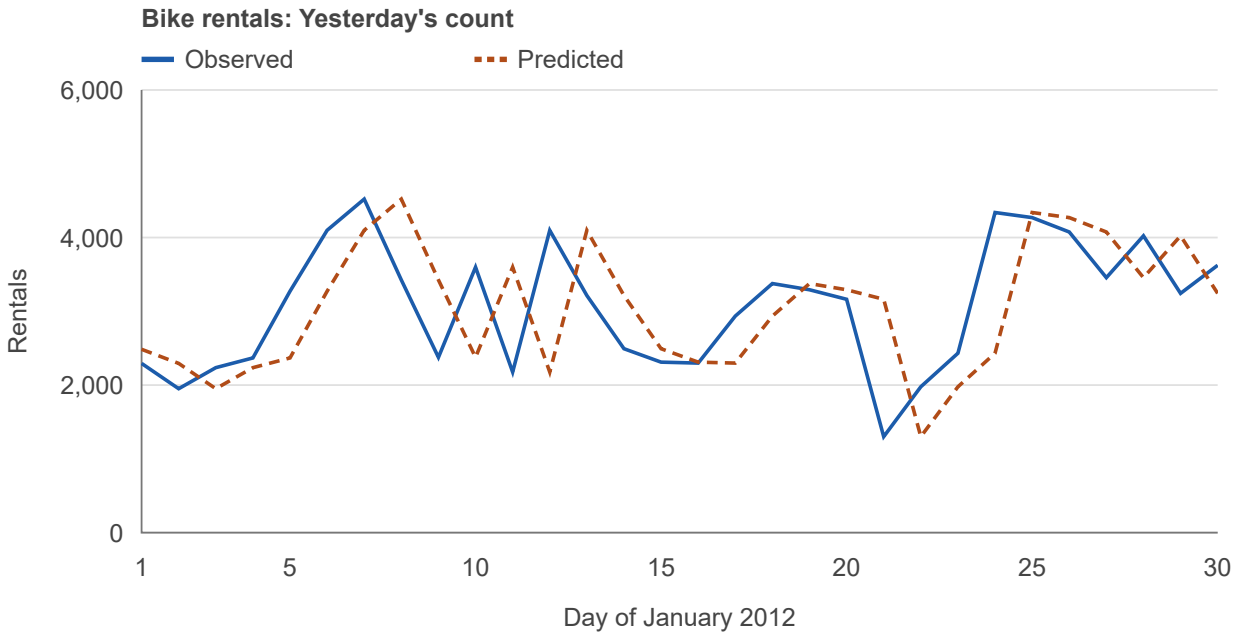


Figure description: The horizontal axis shows January 1 through 30, 2012; the vertical axis shows zero to 6,000 rentals. The solid line is observed demand and the dashed line is the prediction. The dashed prediction line follows the previous day's observed count, so it reproduces changes one day later. Demand fluctuates substantially, with peaks and drops that the prediction does not always match. On January 1 the observed count is 2,294 and the prediction is 2,485.00. Across all 182 validation days, mean absolute error is 900.60 rentals and mean squared error is 1,621,703.66 squared rentals. The comparison table below gives both measures for all three models.

Observed and predicted daily rentals during the first 30 validation days. The reported errors use all 182 validation days, not just this plotted window.

Same validation dates, two error measures

Model	MAE (rentals)	MSE (rentals^2)
Training mean	2,048.42	5,978,763.96
Yesterday's count	900.60	1,621,703.66
Fitted linear model	936.70	1,520,025.84

Both alternatives improve substantially on the training mean in this split. Yesterday's count has the lower MAE, while the fitted linear model has the lower MSE. There is no contradiction: the measures assign different costs to the misses. “Better” needs a stated criterion, and this ranking is a result for this experiment, not a universal ranking of the model families.

7. How closely should a model follow the data?

A constant model can move up or down, but cannot respond to changes in its input. A line has more freedom: it can also change slope. A quadratic can bend, and a higher-degree polynomial can bend in more ways. This is a concrete notion of **model flexibility**: the range of patterns a model family can represent.

$$\hat{y} = c\,; \qquad \hat{y} = a_0 + a_1 x\,; \qquad \hat{y} = a_0 + a_1 x + a_2 x^2\,; \qquad \ldots$$

In words: A constant model predicts c. A linear model predicts a zero plus a one times x. A quadratic model adds a two times x squared. Higher-degree polynomials add higher powers of x with their own coefficients.

A degree-d polynomial has $d + 1$ coefficients, including its constant term. Increasing degree enlarges the family. The coefficients are learned during fitting; the degree is a setting we can choose through validation.

Remembering every detail.

The discussion returned to human learning: do we preserve every small detail, or compress many examples into a broader rule? The useful machine-learning question is whether a detail helps predict new cases. Reproducing the training examples more closely can mean learning their accidental variations as well as their shared pattern.

The next experiment uses synthetic points, separate from the bike-rental records. Every candidate is fitted to the same 14 training points and compared on the same 50 validation points. Change the polynomial degree and inspect both the curve and its errors.

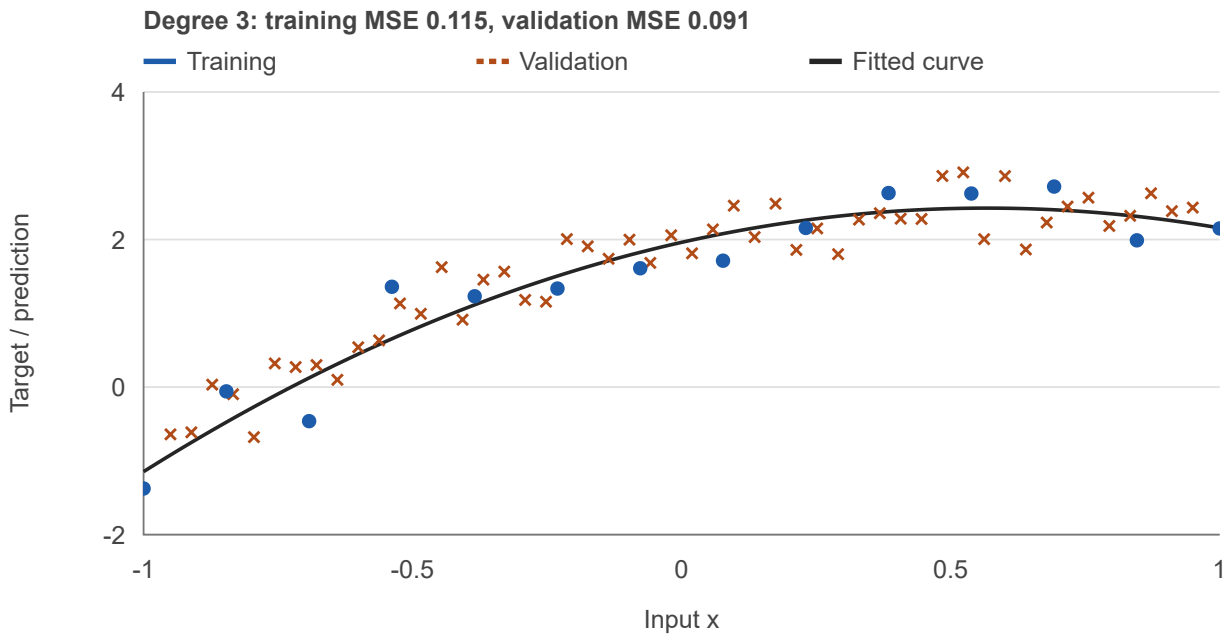


Figure description: The horizontal axis is input x from minus one to one; the vertical axis is target or prediction from minus two to four. Fourteen filled circles are training observations. Fifty crosses are validation observations. The dark curve is a degree 3 polynomial fitted only to the circles. The fitted curve captures the broad rising pattern, which flattens and bends down near the right edge. Training mean squared error is 0.11495; validation mean squared error is 0.09140. All degrees use the same observations and axes.

Blue circles: training observations. Orange crosses: validation observations. The dark curve is fitted only to the blue points. The plot keeps the same vertical scale; extreme oscillations can extend outside it.

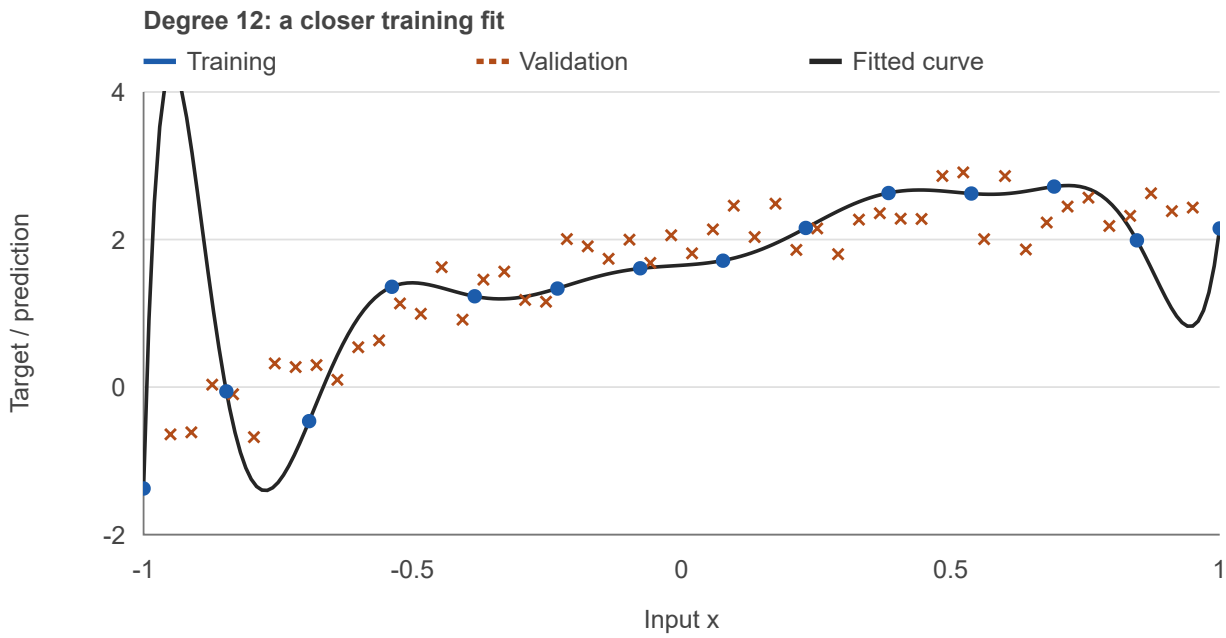


Figure description: The horizontal axis is input x from minus one to one; the vertical axis is target or prediction from minus two to four. Fourteen filled circles are training observations. Fifty crosses are validation observations. The dark curve is a degree 12 polynomial fitted only to the circles. The fitted curve has extra oscillations between training points, particularly near the edges; some exceed the displayed vertical range. Training mean squared error is less than 0.00001; validation mean squared error is 1.13666. All degrees use the same observations and axes.

Degree 12 on the same points: training MSE 0.0000, validation MSE 1.1367. The extra flexibility follows training details but predicts the held-out points less well.

Too little flexibility can miss the main pattern: **underfitting**. Too much can follow training-specific variation that does not recur on new observations: **overfitting**. A small training error alone cannot distinguish a useful pattern from that kind of close imitation.

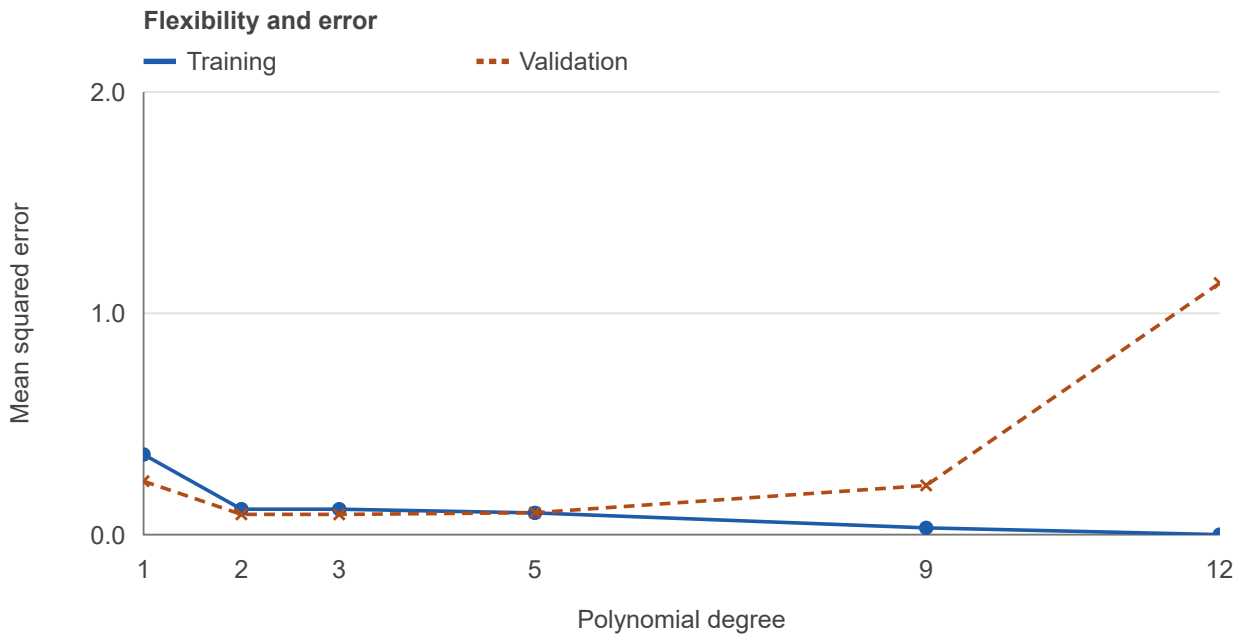


Figure description: The horizontal axis gives polynomial degrees 1, 2, 3, 5, 9, and 12; the vertical axis gives mean squared error. Training uses circles and a solid line; validation uses crosses and a dashed line. Training error falls toward zero. Validation error first falls, reaches its minimum at degree 3, then rises, especially at degree 12. Degree 1: training 0.36216, validation 0.24150; Degree 2: training 0.11496, validation 0.09164; Degree 3: training 0.11495, validation 0.09140; Degree 5: training 0.09823, validation 0.09911; Degree 9: training 0.03055, validation 0.22219; Degree 12: training 0.00000, validation 1.13666. A closer fit to training observations can therefore predict held-out observations less accurately.

Measured training and validation MSE for the six polynomial degrees. The validation minimum is at degree 3 in this experiment.

Training error generally falls as the fitted family becomes more flexible. For nested polynomial families, the exact minimum training squared error cannot increase when we add a degree, because the old solution remains available by setting the new coefficient to zero. Numerical calculations can introduce tiny deviations. Validation error has no such guarantee: here it falls initially, then rises sharply.

Select the candidate with the lowest validation error under the chosen metric. Only after this selection do we assess the final procedure on the reserved test set. Repeatedly searching more candidates can also adapt our choices to the validation sample, which is another reason to keep final test evidence separate.

8. Leakage: when the answer crosses the boundary

Data leakage occurs when information enters fitting or model development that the intended prediction should not use. The resulting score can look excellent while answering the wrong question. The closing examples showed two ways this happens.

Information recorded after the outcome

For an early hospital-risk prediction, a diagnosis code finalized after the hospital stay may reveal part of the outcome. Ramadan and colleagues examined this problem in clinical prediction research: highly informative final codes included brain death. Such a record can be useful for describing a completed stay

while being unavailable for the earlier prediction. The issue is the timing of the information, not just whether different patients appear in different splits. [1]

The same distinction is visible in our rental data. On January 1, 2012, the completed report records 686 casual rentals and 1,608 registered rentals. Their sum is exactly the target, 2,294. Adding these columns reconstructs the final answer perfectly, but does not forecast it at the start of the day. A chronological split cannot make these future inputs available earlier.

A test example already present in training

Barz and Denzler documented exact and near-duplicate images across the training and test portions of CIFAR image-recognition datasets. Recognizing a picture already encountered during training provides weaker evidence about recognizing a new picture. Their ciFAIR datasets replace duplicate test images. The lesson is to check the examples themselves, not merely that the row numbers on the two sides differ. [2]

The exam analogy is useful: answering a previously studied question can demonstrate recall, but it is a different evaluation from solving a genuinely new question. Whether that is appropriate depends on the intended claim. For our models, the evaluation must match the new cases we expect to face.

> **The boundary covers preparation too.**
>
> A mean used to fill missing values, a scaling factor, or a choice of feature columns can be learned from data. Fit such preparation on the training portion, then apply the resulting rule to held-out inputs. Keeping answers out of the final prediction call is insufficient if they have already influenced an earlier step.

A useful result states what was predicted, when the prediction was made, which inputs were available, how the data were split, and how the model compared with a baseline under a named error measure. Those details make a score interpretable. Without them, even a small error is not yet convincing evidence of learning.

Check your understanding

1. What does `fit` receive that `predict` does not? What remains stored after fitting a mean baseline?
2. Why can a column in a historical file be an invalid feature for a morning forecast?
3. Why is fitting on January 3 and validating on January 2 unsuitable for our forecasting task?
4. For targets 2 and 6 and predictions 4 and 4, calculate the mean residual, MAE, and MSE. Which calculation hides the mistakes?
5. Why can yesterday's count beat a fitted linear model under MAE but lose under MSE?
6. If training error falls while validation error rises, what does that suggest about the model's flexibility?
7. How do the hospital and duplicate-image examples cross the prediction boundary in different ways?
8. What changes about the test set's role if its results determine which model you select?

Sources and data

The bike example uses the UCI Bike Sharing dataset, Hadi Fanaee-T, distributed under CC BY 4.0. It contains 731 daily records from 2011–2012; adding yesterday's count leaves 730 usable rows. The constant example and polynomial experiment are synthetic. Their figures and the rental comparisons use the meeting's numerical data.

1. Ramadan et al. (2025), Diagnostic Codes in AI Prediction Models and Label Leakage of Same-Admission Clinical Outcomes.
2. Barz and Denzler (2020), Do We Train on Test Data? Purging CIFAR of Near-Duplicates.
3. scikit-learn documentation: DummyRegressor, regression metrics, and leakage and preprocessing.